

# **OPTIMAL RELATIVE POSE FROM FEATURELESS POINTS WITH ONE KNOWN CORRESPONDENCE**

[redacted]

A THESIS SUBMITTED FOR THE DEGREE OF MASTER OF COMPUTING IN  
COMPUTER SCIENCE

DEPARTMENT OF COMPUTER SCIENCE  
NATIONAL UNIVERSITY OF SINGAPORE

2019

## Declaration

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

---

[redacted]

August 2019

## Acknowledgments

First, I would like to thank my supervisor, Prof. Ng Teck Khim for his invaluable guidance, insight and patient support during my thesis journey. I have enjoyed and benefited greatly from all our shared discussions over the past year. Without his encouragement and advice at critical points, this thesis would not have seen light of day.

I also wish to thank the members of my dissertation committee in advance, for generously offering their time to read and critique my work.

Finally, I am grateful to family, friends and colleagues for their understanding, support, and gifts of food over the year I spent on this project. Thank you to all who made it possible.

## Abstract

Relative pose determination between two camera views is a classical problem in computer vision. Typical approaches to the relative pose problem first require finding a set of point correspondences, before applying solvers combined with outlier detection to recover the epipolar geometry.

This work references and builds on an alternative approach for optimal relative pose estimation with unknown correspondences [7], which uses both appearance and geometrical consistency to recover the point correspondences and epipolar geometry simultaneously. We consider the special case of having one to two known correspondences, and describe a different way to exploit this information, in order to reduce the translation search space by approximately one dimension per correspondence. Improvements in runtime were verified experimentally with synthetic data, and the algorithm was shown to work for real scenes with no outlier points. As runtimes improve, practical applications may benefit as optimal and geometrical methods are brought closer to general use.

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>1</b>  |
| 1.1      | Related work                                                     | 1         |
| 1.2      | The Relative Pose Problem                                        | 3         |
| <b>2</b> | <b>Translation Estimation with Known Rotation</b>                | <b>5</b>  |
| 2.1      | Converting point pairs to geometric constraints on $t$           | 5         |
| 2.2      | Branch and bound                                                 | 7         |
| 2.2.1    | Dividing translation space                                       | 8         |
| 2.2.2    | Upper and lower bounds                                           | 9         |
| 2.2.3    | Stopping conditions                                              | 11        |
| 2.2.4    | Priority queue                                                   | 11        |
| 2.3      | Comparison of patch shapes                                       | 11        |
| 2.3.1    | Experiments                                                      | 12        |
| <b>3</b> | <b>Full Relative Pose</b>                                        | <b>13</b> |
| 3.1      | Representing and dividing rotation space [11]                    | 13        |
| 3.2      | Upper and lower bounds                                           | 13        |
| 3.2.1    | Upper bound                                                      | 14        |
| 3.2.2    | Lower bound                                                      | 14        |
| <b>4</b> | <b>Full relative pose with at least one known correspondence</b> | <b>15</b> |
| 4.1      | Reducing translation search space                                | 15        |
| 4.2      | Dividing the search region into patches                          | 16        |
| 4.3      | Experiments                                                      | 17        |
| 4.3.1    | Comparison of known-correspondence speed-up methods              | 17        |
| 4.3.2    | Increasing the number of scene points                            | 19        |
| 4.3.3    | Evaluation on real data                                          | 20        |
| 4.3.4    | A comment on speed                                               | 21        |
| <b>5</b> | <b>Conclusion</b>                                                | <b>22</b> |
|          | <b>References</b>                                                | <b>23</b> |

# CHAPTER 1

## Introduction

Estimating the relative pose between two camera views is a classical problem in computer vision. It is an important subproblem in most structure-from-motion pipelines for building an overall 3D reconstruction, and has applications in a wide range of tasks, such as object recognition, scene understanding and camera self-calibration. Typical approaches to the relative pose problem first require finding a set of point correspondences, before applying solvers combined with outlier detection to recover the epipolar geometry.

This work references and builds on a recent track of work on optimal relative pose estimation with unknown correspondences [7], which uses both appearance and geometrical consistency to recover the point correspondences and epipolar geometry simultaneously. Their approach uses appearance-based heuristics to filter possible correspondences, then finds a geometrically viable set of matches, using a branch-and-bound method in two levels to search rotation and translation space, combined with maximum-cardinality bipartite matching to compute bounds.

We consider the special case of having one to two known correspondences, and describe a different way to exploit this information, in order to reduce the translation search space by approximately one dimension per correspondence. Improvements in runtime were verified experimentally using synthetic datasets.

### 1.1 Related work

In feature-based relative pose estimation, one of the first major components is the point correspondence problem, which is to identify feature points in the two images that correspond to the same 3D point. In [3], Bazin et al. highlight that feature matching is typically based on two principles: appearance similarity and geometric consistency. Existing correspondence and pose estimation methods differ based on how they exploit these two principles.

Typical approaches isolate the point correspondence problem as an appearance-based matching in the first step, before estimating geometry separately. Common solutions extract features and match them based on proximity and similarity of their intensity neighbourhood, and then filter out poor matches with heuristics, such as only allowing mutual best matches

and matches with unique appearance [15] [2].

Then in the second step, the correspondences are passed as input to solvers that recover the relative pose. The relative pose is commonly and compactly represented as a rank-2,  $3 \times 3$  matrix (the essential/fundamental matrix, for calibrated/uncalibrated cameras respectively) [8]. Recovering this matrix is equivalent to recovering the two-view geometry between the two images. Many algorithms to do this have been proposed over the past decades, and these are broadly divided into three classes (following [1]):

- **Linear methods**, which recover a matrix given at least a minimal number of correspondences. Among the most well-known minimal solvers are Nistér’s five-point solver for the calibrated case [17], and the 7-point ([9]) and 8-point ([14] [10]) algorithms for the uncalibrated case [9]. If it is not implicit in the solver, the rank-2 constraint can be enforced in a few ways, for instance by zeroing out the smallest singular value of the matrix [10].
- **Iterative methods** [1] [22] [21]: Linear methods generally minimise an algebraic error with respect to the matrix, which is not geometrically meaningful, and can depend on the location of the correspondences. If geometric errors are used instead, they tend to require the matrix to be estimated first, which also requires the error to be estimated. This leads to the use of iterative methods, such as iteratively reweighted least squares and gradient-based methods (e.g. Newton-Raphson or Levenberg-Marquardt). Common geometric metrics used are the point-to-epipolar-line distance, the Sampson distance, and reprojection error. For better estimates, the rank-2 constraint can also be enforced between iterations.
- **Robust methods**, which address the common problem of outliers that affect the solution quality. A common approach is to combine minimal solvers with robust estimation methods that enforce geometric consistency, such as RANSAC [5] or least median of squares [19].

The approaches discussed above rely solely on appearance in the initial point correspondence stage, and usually discard points with multiple matches. However, this may not work well in scenes with many visually-similar features, such as those with repetitive textures or low lighting. Recently, there have been alternative approaches that handle multiple possible matches, by incorporating geometric constraints when finding correspondences. In [7], a branch-and-bound framework is used to search over rotation and translation space for a maximal set of correspondences that satisfy the epipolar constraint, building on earlier foundational work in [11], [12] and [6]. A similar alternative approach in [3] uses branch-and-bound with integer programming to find correspondences that satisfy an algebraic error function up to a tolerance.

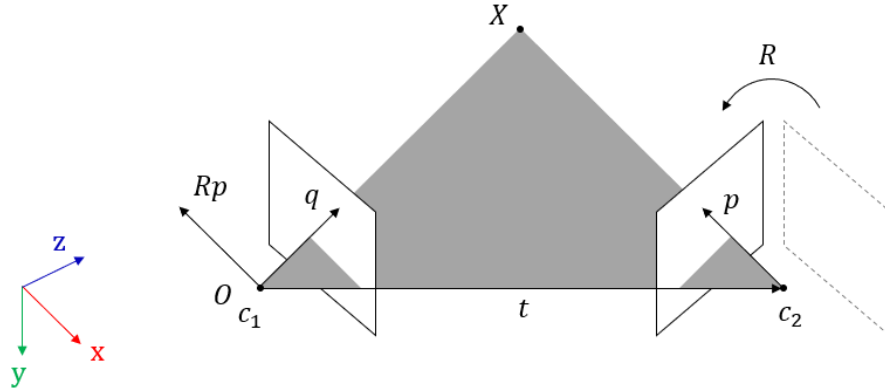
A variety of solutions have been presented for the case of completely unknown correspondences. Two works address this problem with frequency domain techniques for spherical and

orthographic views, given two sets of features [16] [13]. Another method in [20] does not use features, and instead does gradient descent in a 5-dimensional parameter space, with a cost function based on variance of intensity differences between sampled points and their epipolar lines. A recent neural net approach combines feature extraction and matrix estimation in an end-to-end solution, taking stereo images as input [18].

## 1.2 The Relative Pose Problem

We consider the relative pose problem with calibrated cameras and unknown correspondences. We are given two images of the same 3D scene taken from different viewpoints, and the goal is to find the relative rotation and translation between the two viewpoints. The 3D scene points are unknown, and we assume interest points can be extracted from each image using established feature detection techniques.

The following image shows a general two-camera set-up with one scene point  $X$ :



**Figure 1.1:** Two-view set-up.

We use right-handed coordinates, where the  $z$ -axis points forward from the camera. The first camera  $c_1$  is set at the origin and aligned to world coordinates, so that it has zero rotation and translation. The relative rotation and translation of the second camera  $c_2$  are denoted as a  $3 \times 3$  rotation matrix  $R$ , and a 3D vector  $t$ . Vectors  $p$  and  $q$  represent two images of the same scene point  $X$ , viewed from  $c_2$ 's and  $c_1$ 's reference frames respectively.  $Rp$  is the point  $p$  after being rotated to  $q$ 's frame of reference.

Assuming noiseless measurements, the central observation in epipolar geometry is that the vectors  $Rp$ ,  $q$  and  $t$  are coplanar for any corresponding  $(p, q)$  pair arising from the same scene point. This is summed up in the equation:

$$(Rp \times q) \cdot t = 0$$

A looser form of the constraint applies to noisy measurements of corresponding image points. Chapter 2 describes this in further detail.

Next, we restate the original definitions and problem statement [7]:

**Definition 1.** For a candidate rotation and translation  $(R, t)$ , inliers are point pairs that satisfy the constraints of the relative pose within an error threshold  $\epsilon$ . For image vectors, we use the angular reprojection error. Therefore, a pair  $(p, q)$  is an **inlier** if there exists some  $X \in \mathbb{R}^3$  such that

$$\angle(q, X) \leq \epsilon \quad \text{and} \quad \angle(Rp, (X - t)) \leq \epsilon$$

**Definition 2.** Let  $P$  and  $Q$  be the two sets of image vectors from cameras  $c_2$  and  $c_1$  respectively. With no constraints, the set of possible correspondences is all point pairings, i.e. the Cartesian product  $P \times Q$ . We allow constraints from appearance information (e.g. visually dissimilar points should not be matched) or geometry (e.g. when left-to-right order of points is broadly preserved in the two views), and define  $E \subseteq P \times Q$  to be the **set of possible correspondences after applying constraints**.

**Definition 3.** We assume that for a matching of points between two images to be feasible, each point in an image matches to at most one point in the other image. This gives us a one-to-one matching constraint: A set of putative matches  $M \subset E$  is **feasible** if each  $i \in P$  appears in at most one pair  $(i, j) \in M$ , and each  $j \in Q$  appears in at most one pair  $(i, j) \in M$ . We will use the terms “feasible” and “one-to-one” interchangeably.

**Problem statement** Given a set of possible correspondences  $E$  and an error threshold  $\epsilon > 0$ , find the relative rotation  $R$  and translation  $t$  which have a maximal set of feasible inlier matches  $M \subset E$ .

# CHAPTER 2

## Translation Estimation with Known Rotation

First, we introduce the case when relative rotation is known and translation is unknown. This is a useful sub-problem of the full relative pose problem discussed in Chapter 3.

**Translation estimation problem** Given a relative rotation  $R$ , a set of possible correspondences  $E$  and an error threshold  $\epsilon > 0$ , find the translation  $t$  which has a maximal set of feasible inlier matches  $M \subset E$ .

Since there is a scale ambiguity, we can assume that  $t$  is of unit length. Thus the solution space of  $t$  is the surface of a unit sphere,  $S^2$  in  $\mathbb{R}^3$ .

To solve this problem, we take the same general approach as [7] and cast it as a branch-and-bound search over a spherical surface of possible translations. Sections 2.1 and 2.2 restate the methods in [7], and subsections 2.2.1 to 2.2.4 describe new modifications - namely, changing the shape of branch-and-bound blocks from spherical triangles to spherical quadrangles, the computation of bounds for such blocks, and implementation details for branch-and-bound. Section 2.3 compares the spherical quadrangle method to the original spherical triangles.

### 2.1 Converting point pairs to geometric constraints on $t$

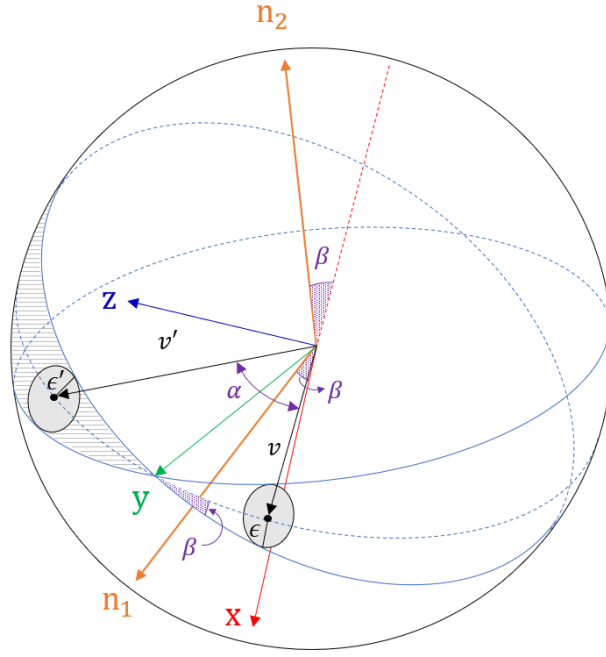
As first described in [12], each point pair in  $E$  forms a geometric constraint on  $t$  in the shape of a spherical wedge. This section repeats the relevant derivations.

Given  $R$ , a point pair  $(p, q)$  forms an epipolar constraint on the translation  $t$ , such that the three vectors  $v = Rp$ ,  $v' = q$ , and  $t$  must be coplanar. If we treat the three coplanar vectors as unit vectors, then  $t$ ,  $v$ , and  $v'$  must all lie on the same great circle on the surface of a unit sphere. We can loosen this constraint to allow for angular reprojection errors  $\epsilon$  in  $p$  and  $\epsilon'$  in  $q$ , so that instead of a great circle,  $t$  now lies within a wedge, formed by two great circles tangent to the error cones around  $v$  and  $v'$ . (See Figure 2.1.)

To find the normals to the two great circles, we need to define basis vectors  $x$ ,  $y$ , and  $z$ .<sup>1</sup>

---

<sup>1</sup> $z$  is defined differently from [12] to follow right-handed coordinates.



**Figure 2.1:** Converting  $v$ ,  $v'$ ,  $\epsilon$ , and  $\epsilon'$  into a wedge constraining  $t$  (shaded in grey).

With reference to the angles in Figure 2.1:

$$\sin^2(\beta) = \frac{\sin^2(\epsilon) + 2\sin(\epsilon)\sin(\epsilon')\cos(\alpha) + \sin^2(\epsilon')}{\sin^2(\alpha)} \quad (2.1)$$

$$y = \frac{\sin(\epsilon)v' + \sin(\epsilon')v}{\sin(\beta)\sin(\alpha)} \quad (2.2)$$

$$x = v \times v' \quad (2.3)$$

$$z = x \times y \quad (2.4)$$

The normals to the two great circles are then:

$$n_1 = \sin(\beta)z + \cos(\beta)x \quad (2.5)$$

$$n_2 = \sin(\beta)z - \cos(\beta)x \quad (2.6)$$

The two great circles form two wedges on the sphere. Only one wedge allows the scene point  $X$  to be in front of both cameras. We can check if  $t$  lies in this wedge by the condition:

$$\angle(n_1, t) \leq \pi/2 \quad \text{and} \quad \angle(n_2, t) \leq \pi/2$$

If a translation  $t$  lies within a wedge, it will have the image pair as an inlier. We can construct one wedge for each possible correspondence in  $E$ , and overlay these wedges on the surface of a unit sphere. Then, the translation estimation problem is equivalent to:

**Translation estimation problem** Given a set of spherical wedges  $W$  formed from the set of possible correspondences  $E$  and error thresholds  $\epsilon > 0$  and  $\epsilon' > 0$ , find the translation  $t$  which lies within a maximal set of wedges  $M_W \subset W$  such that the corresponding matching  $M \subset E$  is feasible.

## 2.2 Branch and bound

Instead of searching continuous  $t$ -space by brute force, we can use a branch-and-bound technique. Branch-and-bound is a general framework frequently used in discrete optimisation problems to find globally optimal solutions. It keeps track of a current best solution as it explores, and for each node representing a set of solutions, it either discards it if it cannot contain solutions better than the current best, or branches and continues searching child nodes. (See Algorithm 2.1.)

---

**Algorithm 2.1:** Branch and bound on block structures.

---

**Result:** Finds a solution with maximal value of objective  $S_t$   
**Input :** Initial list of block structures that divide search space, *block\_list*  
 Stopping threshold for block size, *thres\_stop*  
**Output:** Maximal value of  $S_t$   
 $T$ , a solution block with lower bound equal to the maximal value of  $S_t$

```

1   $S_t = 0$ 
2   $T = \emptyset$ 
3  queue = Queue(block_list)
4
5  while queue is not empty and  $S_t < \text{queue.max\_UB}$  do
6    block = queue.dequeue()
7    block.LB = findLowerBound(block)
8    block.UB = findUpperBound(block)
9
10   if block.LB >  $S_t$  then // Update  $S_t$ , record solution if current best
11      $S_t = \text{block.LB}$ 
12      $T = \text{block}$ 
13     queue.delete(blocks where block.UB <  $S_t$ )
14   end
15
16   if block.size > thres_stop and block.UB >  $S_t$  then
17     // Continue to search if block may contain a better solution
18     subblocks = block.subdivide()
19     queue.enqueue(subblocks)
20   end
21
22  return  $S_t, T$ 
```

---

Applying this framework to our problem, we first define the objective to be maximised. Let the objective  $S_t$  be the size of the maximal set of wedges that represent a feasible matching for a given translation  $t$ . Our goal is to search the surface of a unit sphere for  $t$  that maximises  $S_t$ . Other key components that need to be defined are:

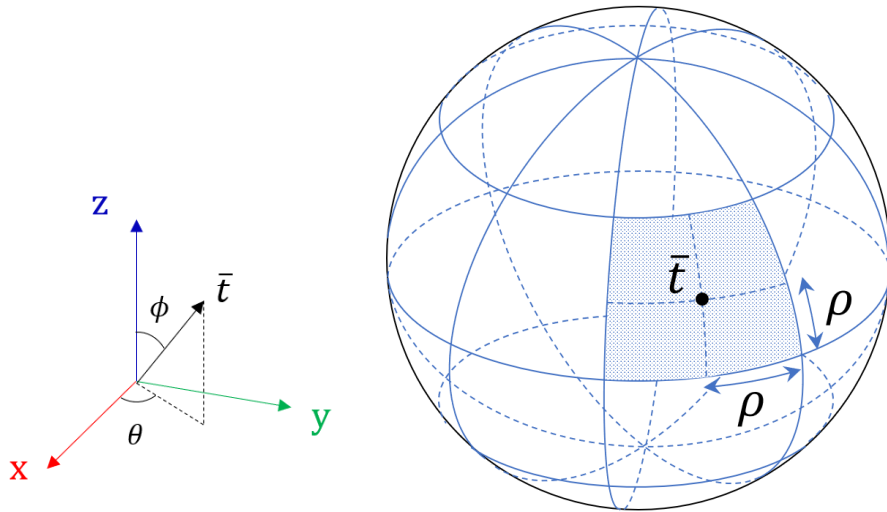
- Blocks that divide the search space, and a method of subdividing blocks
- Functions to derive upper and lower bounds on  $S_t$  for each block
- Stopping conditions: Conditions for termination or convergence, and (optionally) a minimum block size at which we stop subdividing

The following sections describe implementation details and new modifications to the original approach in [7].

### 2.2.1 Dividing translation space

In the branch-and-bound approach, we divide translation space into patches on the surface of a unit sphere, representing groups of adjacent translations. Fredriksson et al. use spherical triangles in their work [6] [7], but here we deviate and use spherical quadrangles instead - four-sided shapes bounded by latitude and longitude lines.

Using the spherical coordinate system, each point on the unit sphere is represented by azimuth angle  $\theta$  and elevation angle  $\phi$ . We can divide the  $\theta$  and  $\phi$  dimensions evenly using the same angular interval for both, to get “square” quadrangles as shown in Figure 2.2. Each patch can be described by its centre,  $\bar{t} = (\theta, \phi)$ , and the patch’s half-length,  $\rho$ , in radians along longitude and latitude lines. When a patch is subdivided, we can halve it along the  $\theta$  and  $\phi$  dimensions to get four sub-patches.



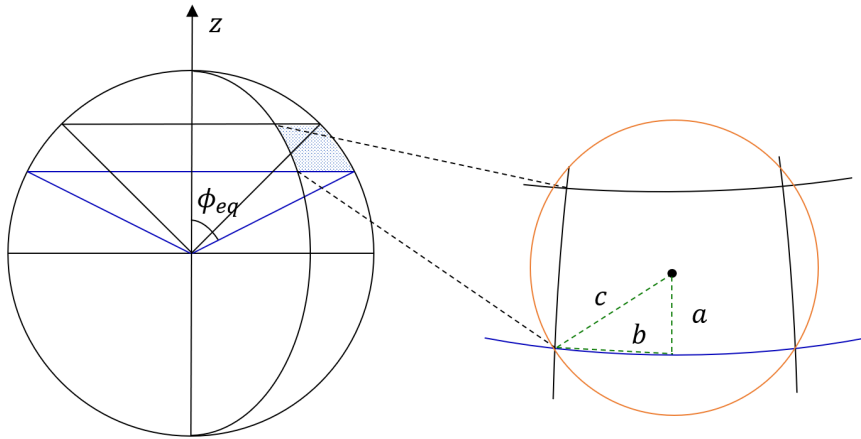
**Figure 2.2:** Spherical quadrangles with sides of equal  $\theta$  and  $\phi$  intervals.

### 2.2.2 Upper and lower bounds

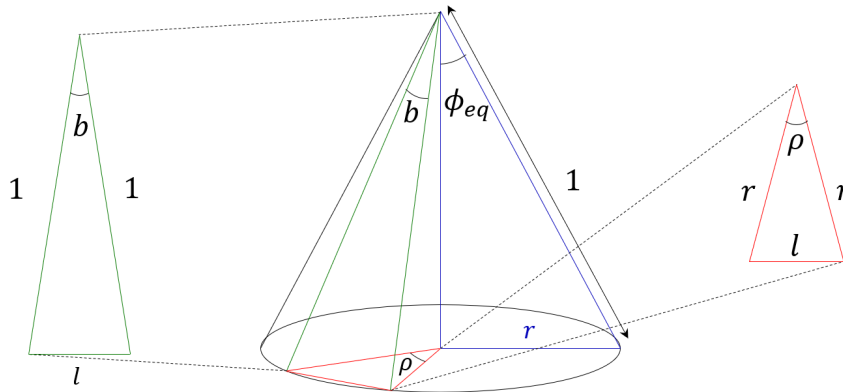
For the spherical triangles used by Fredriksson et al., the upper bound was the sum of wedges intersecting a spherical triangle, and the lower bound was the maximum bipartite matching of wedges intersecting the triangle's centre point [7]. For our spherical quadrangle method, we chose to do a maximum bipartite matching of wedges for both bounds. For the upper bound, we evaluate a region circumscribing the whole patch, and consider all wedges intersecting this region; for the lower bound, we evaluate a smaller region around the patch centre. Details are described below.

#### Upper bound

We aim to circumscribe each patch and check if a wedge intersects this region. In any patch, the point furthest from the patch centre is at the corner where the latitude line nearer the equator intersects either longitude line. We denote the elevation angle at this latitude line  $\phi_{eq}$ .



(a) A spherical quadrangle. Arcs  $a$ ,  $b$ , and  $c$  form a spherical triangle. The orange circle indicates a cone that encloses the quadrangle, with its apex at the centre of the sphere.



(b) Cone formed from a latitude circle, with its apex at the centre of a unit sphere.

**Figure 2.3:** Circumscribing a spherical quadrangle

We can show that for each patch of half-length  $\rho$ , the point furthest from the centre is at an angular distance of  $\cos^{-1}(\cos(\rho) \cos(2 \sin^{-1}(\sin(\phi_{eq}) \sin(\rho/2))))$ . We denote this angular distance  $c$ , which defines a cone around the patch. For the largest patches touching the equator,  $\phi_{eq}$  is  $\pi/2$  and this distance expression simplifies to  $\cos^{-1}(\cos(\rho)^2)$ .

*Proof.* Figure 2.3a shows a spherical quadrangle. The distance we are interested to find is  $c$ . The arcs  $a$ ,  $b$ , and  $c$  are great-circle arcs and form a spherical triangle. From the spherical law of cosines, we have:

$$\cos c = \cos a \cos b$$

We know  $a = \rho$ , and  $b$  is shorter than  $\rho$ , since the patch half-length along a latitude is a small-circle distance. From Figure 2.3b, we can find  $b$  and then  $c$  as follows:

$$r = \sin \phi \tag{2.7}$$

$$l = 2r \sin \frac{\rho}{2} \tag{2.8}$$

$$b = 2 \sin^{-1}\left(\frac{l}{2}\right) \tag{2.9}$$

$$= 2 \sin^{-1}\left(r \sin \frac{\rho}{2}\right) \tag{2.10}$$

$$= 2 \sin^{-1}\left(\sin \phi \sin \frac{\rho}{2}\right) \tag{2.11}$$

$$\therefore \cos c = \cos \rho \cos(2 \sin^{-1}(\sin \phi \sin \frac{\rho}{2})) \tag{2.12}$$

□

To check if a wedge intersects this cone, we check the two wedge normals against the patch centre  $\bar{t}$ , using dot products to find the angles:

$$\angle(n_1, \bar{t}) \leq \pi/2 + c \quad \text{and} \quad \angle(n_2, \bar{t}) \leq \pi/2 + c$$

The above check determines whether a wedge, representing a point-pair, is an inlier or not for the patch upper bound.

To find the largest one-to-one matching given a set of inlier point pairs, we use the well-known maximum bipartite matching method. We construct a bipartite graph whose nodes are the image points in the two images. For each inlier wedge, we add an edge between its pair of points. We then use an established method to find a maximum-cardinality matching in this graph. In our implementation, we use Matlab's maxflow function with the Boykov-Kolmogorov algorithm. The size of the returned maximum bipartite matching is the upper bound on  $S_t$  for any  $t$  in a given patch.

### Lower bound

Our implementation of the branch-and-bound search stops subdividing patches at a user-defined half-length of  $\rho_{stop}$ . In line with this, we evaluate the lower bound of each patch by considering a small cone around the patch centre, defined by angular interval  $c_{stop} = \cos^{-1}(\cos(\rho_{stop})^2)$ . This is the interval that encloses the largest stopping-size patch.

We test if each wedge intersects this cone:

$$\angle(n_1, \bar{t}) \leq \pi/2 + c_{stop} \quad \text{and} \quad \angle(n_2, \bar{t}) \leq \pi/2 + c_{stop}$$

Similar to the upper bound, we construct a bipartite graph and add edges for each intersecting wedge. We return the size of the maximum-cardinality matching as the lower bound on  $S_t$  for any  $t$  in a given patch.

Since the lower bound is computed before the upper bound, there are two cases where we can skip computation for the upper bound and equate the two:

- When patch size reaches stopping threshold ( $\rho \leq \rho_{stop}$ ).
- When patch lower bound reaches the maximum upper bound of all patches still in the queue, *queue.max\_UB*.

### 2.2.3 Stopping conditions

Branch-and-bound stops when there are no more blocks in the queue, or at the convergence condition, when the current best lower bound  $S_t$  reaches the maximum upper bound of all blocks still in the queue, *queue.max\_UB*. Fredriksson et al. explore setting an optimality gap for early termination, where convergence is reached if  $queue.max\_UB - S_t = gap$  [7]. We also use a stopping threshold size to determine when to stop subdividing blocks.

### 2.2.4 Priority queue

In our implementation, the queue of blocks is a priority queue, sorted by the upper bound of each block's parent block. This ensures that blocks with the highest current upper bound are evaluated first.

## 2.3 Comparison of patch shapes

Some differences when using spherical quadrangles instead of spherical triangles are:

- Speed-precision tradeoff in upper bound computation: When checking if a wedge is an inlier for a patch, using spherical quadrangles requires two dot products per wedge, instead of six cross products per wedge in [6]. However, due to the circumscribing of the spherical quadrangles, the upper bound is not as tight as with spherical triangles.

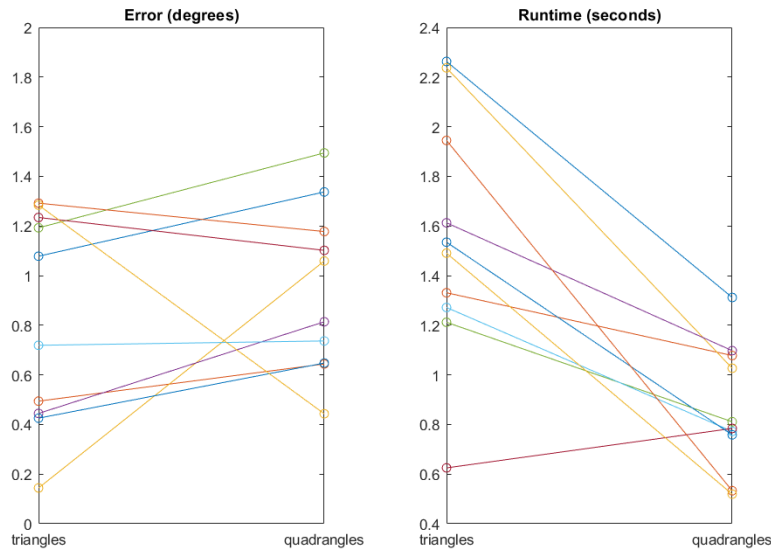
- More restrictions in shape for spherical quadrangles: The sides of the spherical quadrangles are aligned to latitude and longitude lines, which is more restrictive for defining an arbitrary region on the sphere’s surface.
- Vectorisation: The dot product check for quadrangles allows the bounds computation to be more easily vectorised over multiple patches. This was implemented to speed up the full rotation and translation search in Chapters 3 and 4.

### 2.3.1 Experiments

Experiments were run over 10 t-search instances to compare the runtimes and errors of the two patch shapes. We synthesised 30 3D points seen from two views, and added Gaussian noise with standard deviation  $0.033^\circ$  to the image points. The points were featureless and possible matches included all pairings. The camera rotation was known, and we searched  $t$ -space to find  $t$  with the largest feasible inlier set.

The control variables were:

- Known R: The t-search was done over wedges formed from a known R with angular reprojection error,  $\delta = 1/256 \pi$ .
- Initial search space: A unit sphere surface, split into 8 equal patches of half-length  $\pi/4$ .
- Minimum patch half-length,  $\rho_{stop} = 1/512 \pi$



**Figure 2.4:** Comparison of errors and runtimes for two patch shapes over 10 instances.

Figure 2.4 shows the differences in angular error and runtime for the two patch shapes. As predicted, there is a speed-precision tradeoff, with most cases showing slightly higher errors and faster runtimes for the spherical quadrangle patches.

# CHAPTER 3

## Full Relative Pose

Now, we revisit the full relative pose problem with unknown rotation, and frame it as a search over rotations in the outer loop, with translation estimation in the inner loop:

**Problem statement** Given a set of possible correspondences  $E$  and an error threshold  $\epsilon > 0$ , find the relative rotation  $R$  where the translation estimation returns  $t$  with a maximal set of feasible inlier matches  $M \subset E$ , of size  $S_t$ .

This section restates the original approach in [7], based on the earlier work on the branch-and-bound search of rotation space derived by Hartley and Kahl [11].

### 3.1 Representing and dividing rotation space [11]

In the axis-angle representation of a 3D rotation, a rotation  $R$  is parameterised by an axis of rotation, and an angle measuring the magnitude of the rotation about the axis. We can convert this to a vector representation  $r$ , with the axis as its direction and angle as its magnitude. The space of all 3D rotations is then a ball  $B_\pi^3$  of radius  $\pi$  in  $\mathbb{R}^3$ , in which each point represents a rotation.

Since we are using the branch-and-bound strategy to search rotation space, we need to divide the full space of rotations  $B_\pi^3$  into groups of rotations, which we either discard or continue searching. Hartley et al. suggested enclosing  $B_\pi^3$  in a cube,  $D = [-\pi, \pi]^3$  in  $\mathbb{R}^3$ , which is easily divided into sub-cubes  $D_i$  [11]. Each cube  $D_i$  can be described by its centre,  $\bar{R}$ , and its “half-length” in radians,  $\sigma$ . When a cube is subdivided during branch-and-bound, we can halve it along its 3 dimensions to form 8 sub-cubes. In cases where we do not need to search the full rotation space, we can also define cubes accordingly to search a smaller space.

### 3.2 Upper and lower bounds

We aim to find upper and lower bounds on  $S_t$  for any cube. Similar to the translation search, for the upper bound, we evaluate the whole cube, and for the lower bound, we evaluate the cube centre. The following subsections describe how bounds are derived.

### 3.2.1 Upper bound

For the upper bound, we account for uncertainty in rotation  $R$  in a cube  $D$ , and allow  $Rp$  to lie in a region around  $\bar{R}p$ . We describe below how this region is determined for any given cube.

In the vector representation of rotation, the Euclidean distance between two points,  $r$  and  $r'$ , is closely related to the angular difference between the two rotations they represent,  $R$  and  $R'$ . Specifically, it is an upper bound, as derived in [11]:

$$\angle(Rv, R'v) \leq \angle(v, R'R^{-1}v) \leq \|r - r'\|$$

The above statement allows us to reason about the set of rotations in a cube  $D$ . Let  $D$  have a centre  $\bar{R}$  and half-length  $\sigma$ . The furthest point from the cube centre is a corner of the cube, at a distance  $\sqrt{3}\sigma$  away. This is an upper bound on  $\angle(Rv, R'v)$  for any rotation  $R$  in the cube. Hence, for any point  $p$  rotated by a rotation  $R$  in  $D$ , the result  $Rp$  lies within a cone defined by an angle  $\sqrt{3}\sigma$  around  $\bar{R}p$ :

$$\angle(Rp, \bar{R}p) \leq \sqrt{3}\sigma$$

Accounting for a reprojection error  $\delta$  in  $p$ , the new cone radius  $\epsilon$  is:

$$\epsilon = \sqrt{3}\sigma + \delta$$

We can now do translation estimation with  $v = \bar{R}p$ ,  $v' = q$ ,  $\epsilon = \sqrt{3}\sigma + \delta$  and  $\epsilon' = \delta$ . The size of the matching returned by this translation estimation is used as the upper bound on  $S_t$  for any  $R$  in  $D$ .

### 3.2.2 Lower bound

For the lower bound, we simply evaluate the point at the centre of the cube  $\bar{R}$ , by doing translation estimation with  $v = \bar{R}p$ ,  $v' = q$ , and both  $\epsilon$  and  $\epsilon'$  set to the angular reprojection error  $\delta$ . The size of the matching returned is used as the lower bound on  $S_t$  for the cube  $D$ .

# CHAPTER 4

## Full relative pose with at least one known correspondence

Thus far, the branch-and-bound approach described in the previous chapters uses both appearance and geometry to estimate point correspondences and the full relative pose. That is, the appearance-related constraints are encoded in the set of possible correspondences  $E$ , which acts as a filter; and the geometrical constraint is that corresponding  $(p, q)$  pairs must satisfy the coplanar constraint on  $Rp$ ,  $q$ , and  $t$  within an error threshold.

In this chapter, we consider a better use of appearance information in the special case of at least one known correspondence. Possible scenarios include cases where specific points are being tracked, or when the set  $E$  contains some unique matches with high confidence.

### 4.1 Reducing translation search space

The  $R$ - $t$  search problem is in 5 dimensions, and can be solved with Nistér’s 5-point solver given 5 known correspondences [17]. One known correspondence should therefore reduce our uncertainty by approximately one dimension, taking into consideration error thresholds.

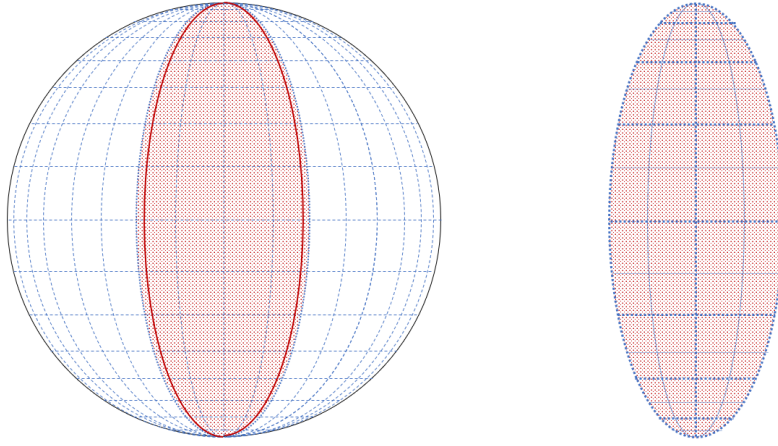
Since any point-pair forms a wedge on the translation search space, the known correspondence forms a wedge as well. The true translation  $t$  that satisfies this constraint must then lie within this wedge. Instead of searching the whole spherical surface in the translation estimation inner loop, we can now restrict our search to just the wedge from the known correspondence.

A similar argument can be made for the case of two or more known correspondences. For two known correspondences, the two wedges representing the point-pairs form an intersection, which must contain the true translation  $t$  satisfying both constraints. We can extrapolate this to any  $n < 5$  number of correspondences forming an intersection of  $n$  wedges. In practice, if the known correspondences can be chosen, it is better to choose points that are far apart in each image, as this will create more dissimilar wedges that have a smaller intersection area.

## 4.2 Dividing the search region into patches

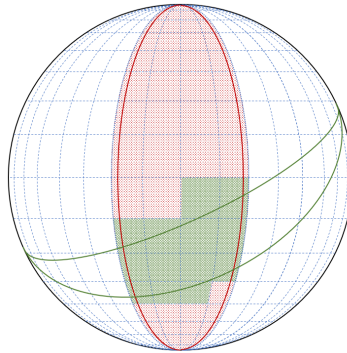
For the case with one known correspondence, the restricted search region is a wedge, and for two known correspondences it is an intersection of wedges. Since the translation estimation step is initialised with a list of spherical quadrangle patches, we need to convert both search regions to patches we can use as input.

A wedge is defined by two normals, and is not necessarily aligned to latitude and longitude lines. To convert the wedge to patches, we rotate the wedge to align its vertices to the  $z$ -axis, then tile the wedge with patches of equal angular intervals in azimuth and elevation, exceeding the wedge slightly if necessary (see Figure 4.1).



**Figure 4.1:** Converting a wedge to patches: The wedge, outlined in red on the sphere, is first covered in patches of half-length  $\pi/32$ . Then patches are repeatedly combined in groups of four as shown on the right, up to the largest patch size that can tile the same area, to form the final list of patches.

For the intersection of wedges, we simply tile one wedge with patches as before, then remove the patches that do not intersect the second wedge. The resulting list of patches fully covers the intersection region (see Figure 4.2).



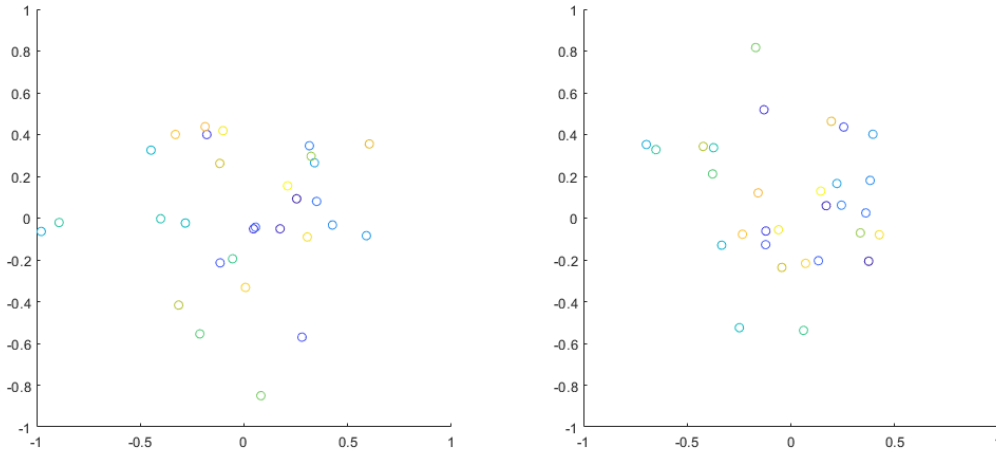
**Figure 4.2:** Converting an intersection of wedges to patches: We select the patches in the first wedge (red) that also lie in the second wedge (green outline). These are shaded in green.

## 4.3 Experiments

### 4.3.1 Comparison of known-correspondence speed-up methods

We set up synthetic data with one to two known correspondences, and compared our method of restricting translation search space to placing the known correspondence(s) in the filter of the set of possible correspondences.

We set up a similar experiment to the narrow field of view (FOV) case in [7]. We synthesised 30 3D points, uniformly distributed in a length-2 cube centred at  $(0,0,2)$ . The points are featureless and possible matches include all pairings, except the known correspondences. The first camera is at the origin and aligned to world coordinates. The second camera is placed randomly at distance 2 from the centre of the cube, in a hemispherical region in the negative  $z$ -direction, and is pointed towards the cube. It is also tilted randomly in the range  $(-\pi/8, \pi/8)$ . We added Gaussian noise with standard deviation  $0.033^\circ$  to the image points, and randomly selected one to two true correspondences as the known correspondences. A sample pair of images is shown in Figure 4.3.



**Figure 4.3:** Example of two views of a scene with 30 points and narrow FOV.

For experimental parameters, the control variables across all experiments were:

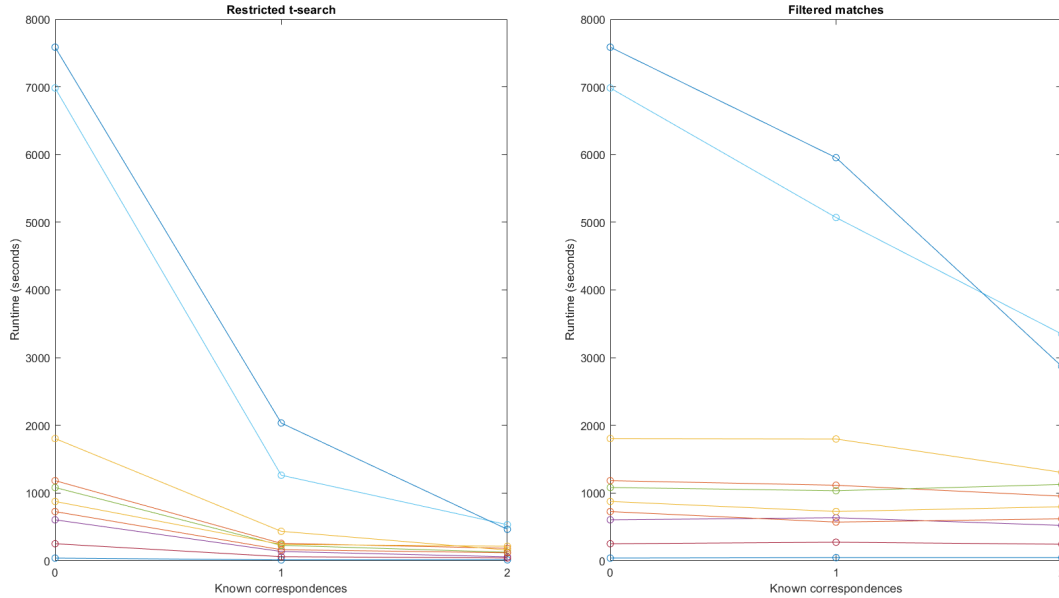
- Initial R search space: The region in axis-angle vector representation,  $x = (-\pi/2, \pi/2)$ ,  $y = (-\pi/2, \pi/2)$ ,  $z = (-\pi/8, \pi/8)$ , split into 16 cubes.
- Angular reprojection error,  $\delta = 0.5^\circ$
- Initial t search space: The entire sphere surface, split into two hemispherical patches.
- Minimum patch half-length,  $\rho_{stop} = 1/512 \pi$

Results for the two methods show nearly identical errors in estimated rotation and translation, as shown in Table 4.1.

**Table 4.1:** Errors over 10 synthetic 30-point narrow FOV instances.

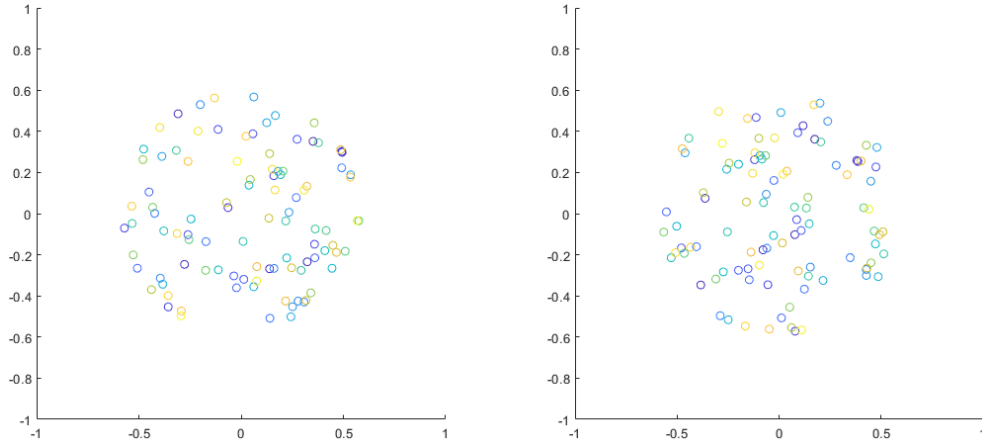
| Method              | Performance | Rotation error | Translation error |
|---------------------|-------------|----------------|-------------------|
| Restricted t-search | Best        | $0.56^\circ$   | $0.63^\circ$      |
|                     | Average     | $4.34^\circ$   | $2.31^\circ$      |
|                     | Worst       | $11.1^\circ$   | $4.59^\circ$      |
| Filtered matches    | Best        | $0.56^\circ$   | $0.26^\circ$      |
|                     | Average     | $4.34^\circ$   | $2.49^\circ$      |
|                     | Worst       | $11.1^\circ$   | $5.39^\circ$      |

Figure 4.4 shows the relative improvements in runtime when using the two methods on 10 synthetic narrow field of view instances, with 0, 1 and 2 known correspondences respectively. Both methods benefit from known correspondences, but the speed-up is greater for the restricted t-search method. Also, for the restricted t-search method, the greatest time savings are made with the addition of the first correspondence (i.e. reducing the search space from the whole spherical surface to a wedge).

**Figure 4.4:** Runtime improvements per scene after adding 0-2 known correspondences, for 10 synthetic 30-point narrow FOV instances

### 4.3.2 Increasing the number of scene points

Similar to the experiment in section 4.3.1, we set up synthetic data with a narrow field of view, but made changes to more closely mimic the real-data scenario of points on a light sculpture. The number of points was increased to 100, and the points were randomly distributed on a spherical surface within the space of the length-2 cube. Other factors remained the same. An example of two views is shown in Figure 4.5.



**Figure 4.5:** Example of two views of a spherical surface with 100 points and narrow FOV.

Experiments were done with the restricted t-search method using two known correspondences, and with the control variables from the previous experiment. A comparison of runtimes and errors for the 30-point and 100-point scenes is shown in Table 4.3.

**Table 4.2:** Runtimes and errors over 10 synthetic narrow FOV instances, using restricted t-search and 2 known correspondences.

| Scene points | Performance | Runtime   | Rotation error | Translation error |
|--------------|-------------|-----------|----------------|-------------------|
| 30           | Best        | 13.5 s    | 0.56°          | 0.63°             |
|              | Average     | 191.2 s   | 4.34°          | 2.31°             |
|              | Worst       | 532.6 s   | 11.1°          | 4.59°             |
| 100          | Best        | 47.77 s   | 2.31°          | 1.38 °            |
|              | Average     | 1361.20 s | 5.60°          | 3.25 °            |
|              | Worst       | 4789.22 s | 10.74°         | 5.79°             |

Results show that runtime has increased significantly with the number of points, while errors are comparable. An increased number of points would have the most effect on the maximum bipartite matching step, which has a theoretical worst case time complexity of  $O(mn^2|C|)$  [4] for a graph with  $n$  nodes,  $m$  edges and size of minimum cut  $|C|$  - in our case, this translates

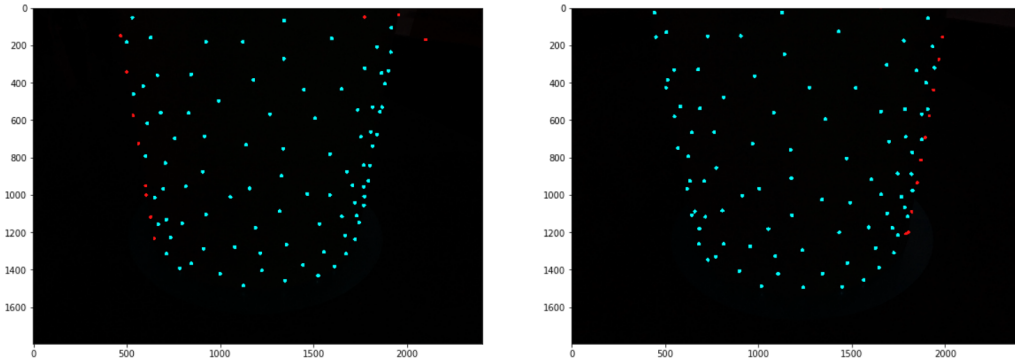
to  $O(n^5)$  for  $n$  scene points.

The above runtime results can give us a sense for the expected performance with a real 100-point scene. For example, the 100-point instance with the worst runtime of 4789.22s ran for 7816 iterations, while the maximum possible number of iterations before termination is 9360 at that resolution, so this is fairly close to worst-case runtime.

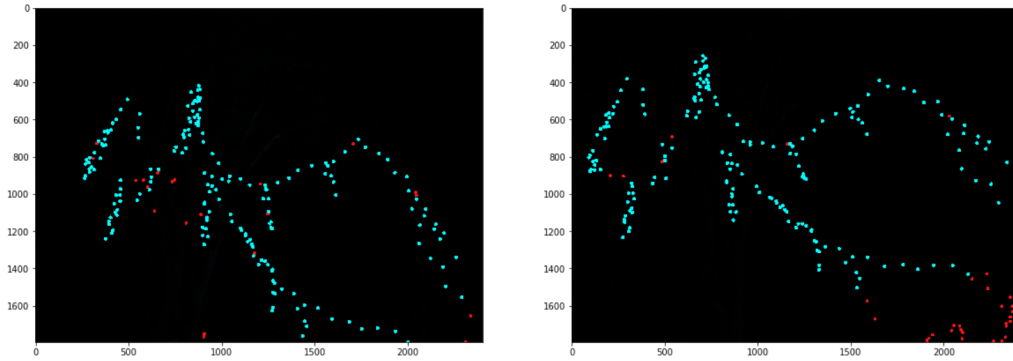
### 4.3.3 Evaluation on real data

The above methods were tested on two real scenes of light displays, photographed with a compact camera in manual mode (i.e. calibrated). Images were taken with low exposure to isolate the points of light. They were then converted to greyscale, and details were removed with box-blurring. Finally, point centres were located by thresholding on intensity and taking the mean of connected components.

Initial experiments showed poor performance when outliers were present, which in our case were the scene points without matches in the other view. As such, experiments were done to compare performance with and without outliers (removed manually). Figure 4.6 shows the two scenes used and the number of points found in each view.



(a) Detected points for a pillar light display. After outliers in red were removed, the number of points were reduced from 101 to 90 in each image.



(b) Detected points for a tree light display. After outliers in red were removed, the number of points were reduced from 204 (left image) and 212 (right image) to 187 in each image.

**Figure 4.6:** Points detected in two scenes, with removed outliers marked in red.

Experimental parameters kept constant were:

- Initial R search space: The region in axis-angle vector representation,  $x = (-\pi/8, \pi/8)$ ,  $y = (-\pi/2, \pi/2)$ ,  $z = (-\pi/8, \pi/8)$ , split into 4 cubes.
- Angular reprojection error,  $\delta = 0.25^\circ$
- Initial t search space: The entire sphere surface, split into two hemispherical patches.
- Minimum patch half-length,  $\rho_{stop} = 1/512 \pi$
- Number of known correspondences provided: 2

Results were compared to ground truth rotation and translation values, obtained manually using SVD and hand-picked correspondences (13 for the pillar scene, 21 for the tree scene). Runtimes and errors are shown in Table 4.3.

**Table 4.3:** Runtimes and errors for real light display scenes

| Scene  | Outliers present | Matches found/<br>True matches <sup>1</sup> | Runtime  | Rotation error | Translation error |
|--------|------------------|---------------------------------------------|----------|----------------|-------------------|
| Pillar | Yes <sup>2</sup> | 99/88                                       | 2019.0 s | 59.28°         | 21.43°            |
|        | No               | 88/88                                       | 1014.1 s | 4.28°          | 6.55 °            |
| Tree   | Yes              | 202/185                                     | 41941.3  | 45.37°         | 98.33°            |
|        | No               | 185/185                                     | 1604.4 s | 1.48°          | 6.01 °            |

Results for the scenes without outliers are similar to the 100-point synthetic case, with comparable errors and runtime close to the synthetic average case. However, the case with outliers has poor performance in both runtime and accuracy. This can be attributed to the way outliers add false matches at various R and t candidate solutions, skewing the upper and lower bounds of the number of matches upward, such that the search is prolonged and/or returns a false solution. From this, we conclude that the algorithm is not robust to outliers.

#### 4.3.4 A comment on speed

Across experiments, the absolute runtimes reported are considerably longer than those in [7], e.g. the average for the 50-point narrow FOV case in [7] is 686s, while our 30-point average is 2113.7s. Further work can be done to explore faster implementations, such as using C++ or optimising computations with a GPU.

<sup>1</sup>With 2 known correspondences, the number of true matches was reduced by 2 in each scene.

<sup>2</sup>Due to long runtimes with the original settings, the search was tweaked to have a minimum size for R-blocks. Blocks were not subdivided after half-lengths below  $1/1024 \pi$ .

# CHAPTER 5

## Conclusion

With reference to existing work on optimal relative pose with unknown correspondences [7], this work has shown a different way to exploit one to two known correspondences for improvements in runtime, through reducing translation search space by approximately one dimension per correspondence. An alternative method of dividing translation search space is also presented, using spherical quadrangles instead of triangles to trade precision for speed. As runtime is a significant factor for the widespread use of non-optimal methods, improved runtimes may help guaranteed-optimal methods to enter into more general use for correspondence and relative pose problems.

From experiments, we also note two main drawbacks of our implementation, namely the lack of robustness to outliers and the long absolute runtimes. Future work can be done to explore methods to handle outliers and faster implementations.

More broadly, with most current approaches using appearance-based correspondences, methods that are less reliant on appearance and incorporate geometrical information open up possibilities for applications where appearance-based matching fails. Examples include low-light scenes, changes in lighting between images, or matching for images of different types (e.g. RGB images and thermal images). Removing the barrier of long runtimes and improving robustness to outliers may have practical gains for applications of this nature.

## References

- [1] X. Armangué and J. Salvi. Overall view regarding fundamental matrix estimation. *Image and vision computing*, 21(2):205–220, 2003.
- [2] H. Bay, T. Tuytelaars, and L. Van Gool. Surf: Speeded up robust features. In *European conference on computer vision*, pages 404–417. Springer, 2006.
- [3] J.-C. Bazin, H. Li, I. S. Kweon, C. Demonceaux, P. Vasseur, and K. Ikeuchi. A branch-and-bound approach to correspondence and grouping problems. *IEEE transactions on pattern analysis and machine intelligence*, 35(7):1565–1576, 2012.
- [4] Y. Boykov and V. Kolmogorov. An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 26(9):1124–1137, 2004.
- [5] M. A. Fischler and R. C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [6] J. Fredriksson, V. Larsson, and C. Olsson. Practical robust two-view translation estimation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2684–2690, 2015.
- [7] J. Fredriksson, V. Larsson, C. Olsson, and F. Kahl. Optimal relative pose with unknown correspondences. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1728–1736, 2016.
- [8] R. Hartley and A. Zisserman. *Multiple view geometry in computer vision*. Cambridge university press, 2003.
- [9] R. I. Hartley. Projective reconstruction and invariants from multiple images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(10):1036–1041, 1994.
- [10] R. I. Hartley. In defence of the 8-point algorithm. In *Proceedings of IEEE international conference on computer vision*, pages 1064–1070. IEEE, 1995.
- [11] R. I. Hartley and F. Kahl. Global optimization through searching rotation space and optimal estimation of the essential matrix. In *2007 IEEE 11th International Conference on Computer Vision*, pages 1–8. IEEE, 2007.

- 
- [12] J.-H. Kim, H. Li, and R. Hartley. Motion estimation for multi-camera systems using global optimization. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8. IEEE, 2008.
  - [13] S. Lehmann, I. V. L. Clarkson, A. P. Bradley, J. Williams, and P. J. Kootsookos. Robust fundamental matrix determination without correspondences. In *Proceedings of the APRS Workshop on Digital Image Computing*. The Australian Pattern Recognition Society, 2005.
  - [14] H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293(5828):133, 1981.
  - [15] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60(2):91–110, 2004.
  - [16] A. Makadia, C. Geyer, S. Sastry, and K. Daniilidis. Radon-based structure from motion without correspondences. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, volume 1, pages 796–803. IEEE, 2005.
  - [17] D. Nister. An efficient solution to the five-point relative pose problem. In *2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2003. Proceedings.*, volume 2, pages II–195. IEEE, 2003.
  - [18] O. Poursaeed, G. Yang, A. Prakash, Q. Fang, H. Jiang, B. Hariharan, and S. Belongie. Deep fundamental matrix estimation without correspondences. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 0–0, 2018.
  - [19] P. J. Rousseeuw. Least median of squares regression. *Journal of the American statistical association*, 79(388):871–880, 1984.
  - [20] S. Roy and I. J. Cox. Motion without structure. In *Proceedings of 13th International Conference on Pattern Recognition*, volume 1, pages 728–734. IEEE, 1996.
  - [21] P. H. Torr and D. W. Murray. The development and comparison of robust methods for estimating the fundamental matrix. *International journal of computer vision*, 24(3):271–300, 1997.
  - [22] Z. Zhang. Determining the epipolar geometry and its uncertainty: A review. *International journal of computer vision*, 27(2):161–195, 1998.